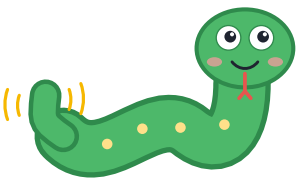
Lecture 5

Part 2: Decisions and Loops

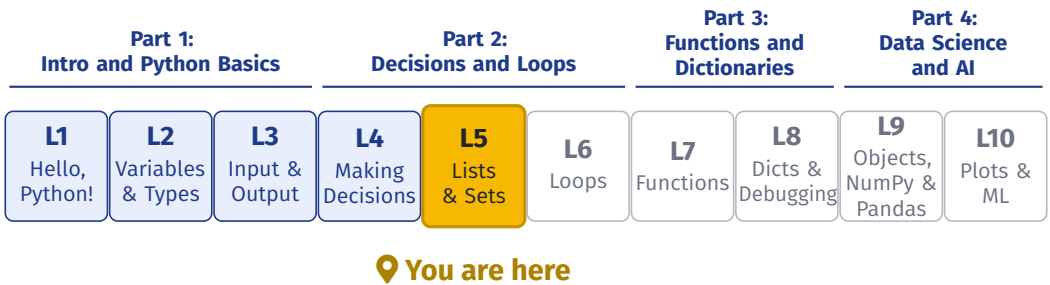
Lists & Sets

Shopping lists, playlists, and groups with no duplicates



Dror Jacoby
Tel Aviv University

Where we are


Part 1:
Intro and Python Basics
Part 2:
Decisions and Loops
Part 3:
Functions and
Dictionaries
Part 4:
Data Science
and AI
L1
Hello,
Python!
L2
Variables
& Types
L3
Input &
Output
L4
Making
Decisions
L5
Lists
& Sets
L6
Loops
L7
Functions
L8
Dicts &
Debugging
L9
Objects,
NumPy &
Pandas
L10
Plots &
ML
You are here

Looking back: lectures 3 and 4

✔ Lecture 3: Input, Output & Conditions

- `input()` always gives text. Convert with `int()`, `float()`
- f-strings fill in the blanks: `f"{price:.2f}"`
- Comparisons give `True`/`False`. `=` stores, `==` asks
- **`if`**/**`else`** picks a path. Indentation means “belongs to”

✔ Lecture 4: Making Decisions

- **`if`** / **`elif`** / **`else`**: the first True wins, order matters
- Combine with **`and`**, **`or`**, **`not`**
- Inside, everything is bits: 1 = True, 0 = False
- `bin()` shows a number in binary

Warm-up: lectures 3 and 4 in one program

```
age = int("17")
price = 12.5
if age < 12:
    fare = 0
elif age < 18 or age >= 67:
    fare = price / 2
else:
    fare = price
print(f"Fare: {fare:.2f} ILS")
```

Warm-up: lectures 3 and 4 in one program

```
age = int("17")
price = 12.5
if age < 12:
    fare = 0
elif age < 18 or age >= 67:
    fare = price / 2
else:
    fare = price
print(f"Fare: {fare:.2f} ILS")
```

OUTPUT

```
Fare: 6.25 ILS
```

Quick check: predict the output

```
x = 15
print(x > 10 and x % 2 == 0)
print(not x == 15 or x < 20)
print(bin(5))
```

Quick check: predict the output

```
x = 15
print(x > 10 and x % 2 == 0)
print(not x == 15 or x < 20)
print(bin(5))
```

OUTPUT

```
False
True
0b101
```

Quick check: predict the output

```
x = 15
print(x > 10 and x % 2 == 0)
print(not x == 15 or x < 20)
print(bin(5))
```

OUTPUT

```
False
True
0b101
```

- **and**: both sides must be True. 15 is odd, so False
- **not** flips only `x == 15`, then **or** finds `x < 20`
- 5 = 4 + 1, so binary `101`

30 students, 30 variables?

```
grade1 = 88
grade2 = 72
grade3 = 95
# ... 27 more lines ...
grade30 = 61
```

- Boring to type

30 students, 30 variables?

```
grade1 = 88
grade2 = 72
grade3 = 95
# ... 27 more lines ...
grade30 = 61
```

- Boring to type
- Hard to find the best grade
- What if 5 more students join?



30 students, 30 variables?

```
grade1 = 88
grade2 = 72
grade3 = 95
# ... 27 more lines ...
grade30 = 61
```

- Boring to type
- Hard to find the best grade
- What if 5 more students join?



We need one box that holds many values.

Today's plan

◎ By the end of today you can…

- Keep many values in a **list** and change it
- Use a **set** to drop duplicates and compare groups
- Grab any item, or a slice, by its position



Today's outline

1 **Lists**

Many values in one variable: add, remove, search, sort, sum, max

2 **Sets**

No duplicates; in both, in either, only in one

3 **Positions: indexing and slicing**

`x[0]`, `x[-1]`, `x[1:4]`, `x[::-1]`, on lists and on text



Every topic: examples, **Try it!** and **Predict the output**. At the end: **Bug Zoo**, **AI Corner**, **Recap**.

Why today matters

Why learn this?

- Real data comes in many values, not one
- One list replaces dozens of separate variables
- Sets answer “who is in both groups?” in one line

Where you will see it

- Playlists, shopping carts, contact lists
- Your feed: a list of posts
- “Mutual friends”: the overlap of two sets





Topic 1

Lists

Lecture 5 • Lists & Sets



TODAY

1 **Lists**

2 Sets

3 Positions: indexing and slicing

A list is a shopping list

L05_01_lists.ipynb

```
shopping = ["milk", "eggs", "hummus", "pita"]
print(shopping)
print(len(shopping))
```

A list is a shopping list

L05_01_lists.ipynb

```
shopping = ["milk", "eggs", "hummus", "pita"]
print(shopping)
print(len(shopping))
```

OUTPUT

```
['milk', 'eggs', 'hummus', 'pita']
4
```

A list is a shopping list

L05_01_lists.ipynb

```
shopping = ["milk", "eggs", "hummus", "pita"]
print(shopping)
print(len(shopping))
```

OUTPUT

```
['milk', 'eggs', 'hummus', 'pita']
4
```

- Square brackets `[ ]`, commas between items
- `len()` counts the items
- Items can be text, numbers, anything

Lists are everywhere

- A Spotify playlist: songs, in order
- A WhatsApp chat: messages, in order
- The line at the bus stop: people, in order
- Your grades this year: numbers, in order

A list keeps its items **in order**, and can change.



append: add to the end

```
shopping = ["milk", "eggs"]
shopping.append("coffee")
shopping.append("bananas")
print(shopping)
```

append: add to the end

```
shopping = ["milk", "eggs"]
shopping.append("coffee")
shopping.append("bananas")
print(shopping)
```

OUTPUT

```
['milk', 'eggs', 'coffee', 'bananas']
```

append: add to the end

```
shopping = ["milk", "eggs"]
shopping.append("coffee")
shopping.append("bananas")
print(shopping)
```

OUTPUT

```
['milk', 'eggs', 'coffee', 'bananas']
```

- Like writing a new line at the bottom of your list
- The dot means “list, please do this”
- Such an action is a **method**. In Lecture 9 you write your own



insert: cut in line

```
queue = ["Noa", "Omer", "Lior"]
queue.insert(1, "Maya")
print(queue)
queue.insert(0, "Savta")
print(queue)
```

insert: cut in line

```
queue = ["Noa", "Omer", "Lior"]
queue.insert(1, "Maya")
print(queue)
queue.insert(0, "Savta")
print(queue)
```

OUTPUT

```
['Noa', 'Maya', 'Omer', 'Lior']
['Savta', 'Noa', 'Maya', 'Omer', 'Lior']
```

insert: cut in line

```
queue = ["Noa", "Omer", "Lior"]
queue.insert(1, "Maya")
print(queue)
queue.insert(0, "Savta")
print(queue)
```

OUTPUT

```
['Noa', 'Maya', 'Omer', 'Lior']
['Savta', 'Noa', 'Maya', 'Omer', 'Lior']
```

- `insert(pos, item)`
- Positions start at **0**: the front of the line
- Everyone behind moves back
- Savta (grandma) always goes first



remove and pop: cross things off

```
shopping = ["milk", "eggs", "pita"]
shopping.remove("eggs")
print(shopping)
last = shopping.pop()
print(last)
print(shopping)
```

remove and pop: cross things off

```
shopping = ["milk", "eggs", "pita"]
shopping.remove("eggs")
print(shopping)
last = shopping.pop()
print(last)
print(shopping)
```

OUTPUT

```
['milk', 'pita']
pita
['milk']
```

remove and pop: cross things off

```
shopping = ["milk", "eggs", "pita"]
shopping.remove("eggs")
print(shopping)
last = shopping.pop()
print(last)
print(shopping)
```

OUTPUT

```
['milk', 'pita']
pita
['milk']
```



- `remove(item)`: remove by **name**
- `pop()`: take the **last** item off and hand it to you

in: is it on the list?

```
shopping = ["milk", "eggs", "tea"]
print("milk" in shopping)
print("pizza" in shopping)
if "pizza" not in shopping:
    print("Add pizza!")
```

in: is it on the list?

```
shopping = ["milk", "eggs", "tea"]
print("milk" in shopping)
print("pizza" in shopping)
if "pizza" not in shopping:
    print("Add pizza!")
```

OUTPUT

```
True
False
Add pizza!
```

in: is it on the list?

```
shopping = ["milk", "eggs", "tea"]
print("milk" in shopping)
print("pizza" in shopping)
if "pizza" not in shopping:
    print("Add pizza!")
```

OUTPUT

```
True
False
Add pizza!
```

- **in** gives `True` or `False`
- Perfect inside an **if** (Lecture 4!)



sort: put it in order

```
names = ["Omer", "Dana", "Lior", "Avi"]
names.sort()
print(names)
grades = [88, 72, 95, 60]
grades.sort(reverse=True)
print(grades)
```

sort: put it in order

```
names = ["Omer", "Dana", "Lior", "Avi"]
names.sort()
print(names)
grades = [88, 72, 95, 60]
grades.sort(reverse=True)
print(grades)
```

OUTPUT

```
['Avi', 'Dana', 'Lior', 'Omer']
[95, 88, 72, 60]
```

sort: put it in order

```
names = ["Omer", "Dana", "Lior", "Avi"]
names.sort()
print(names)
grades = [88, 72, 95, 60]
grades.sort(reverse=True)
print(grades)
```

OUTPUT

```
['Avi', 'Dana', 'Lior', 'Omer']
[95, 88, 72, 60]
```

- Text: A to Z
- Numbers: small to big
- `reverse=True`: big first



sum, max, min: my coffee week

```
coffee = [14, 18, 0, 22, 14]
print(sum(coffee))
print(max(coffee), min(coffee))
print(sum(coffee) / len(coffee))
```

sum, max, min: my coffee week

```
coffee = [14, 18, 0, 22, 14]
print(sum(coffee))
print(max(coffee), min(coffee))
print(sum(coffee) / len(coffee))
```

OUTPUT

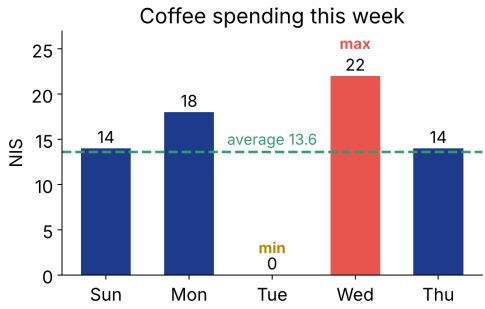
```
68
22 0
13.6
```

sum, max, min: my coffee week

```
coffee = [14, 18, 0, 22, 14]
print(sum(coffee))
print(max(coffee), min(coffee))
print(sum(coffee) / len(coffee))
```

OUTPUT

```
68
22 0
13.6
```



Average = total ÷ how many days.

Example: a grade report in 4 lines

```
grades = [88, 72, 95, 60, 91]
average = sum(grades) / len(grades)
print(f"Students: {len(grades)}")
print(f"Average: {average:.1f}")
print(f"Best: {max(grades)}, worst: {min(grades)}")
```

Example: a grade report in 4 lines

```
grades = [88, 72, 95, 60, 91]
average = sum(grades) / len(grades)
print(f"Students: {len(grades)}")
print(f"Average: {average:.1f}")
print(f"Best: {max(grades)}, worst: {min(grades)}")
```

OUTPUT

```
Students: 5
Average: 81.2
Best: 95, worst: 60
```

Example: a grade report in 4 lines

```
grades = [88, 72, 95, 60, 91]
average = sum(grades) / len(grades)
print(f"Students: {len(grades)}")
print(f"Average: {average:.1f}")
print(f"Best: {max(grades)}, worst: {min(grades)}")
```

OUTPUT

```
Students: 5
Average: 81.2
Best: 95, worst: 60
```



Add a student: nothing else changes!

Try it!

Try it in Colab notebooks/L05/L05_01_lists.ipynb

1. Make a list of 4 songs you love. Print it and its length
2. Append one song, remove another, then sort the list
3. Make a list of your last 5 grades. Print the average



Predict the output

Predict the output!

```
todo = ["laundry", "gym"]
todo.append("study")
todo.insert(0, "coffee")
todo.remove("laundry")
print(todo)
print(len(todo))
```

Predict the output

Predict the output!

```
todo = ["laundry", "gym"]
todo.append("study")
todo.insert(0, "coffee")
todo.remove("laundry")
print(todo)
print(len(todo))
```

Answer

```
['coffee', 'gym', 'study']
3
```

Coffee goes to the front, study to the end, laundry is gone.

Topic 2

Sets

Lecture 5 • Lists & Sets



TODAY

1 Lists

2 Sets

3 Positions: indexing and slicing

A set: nobody is in twice

L05_02_sets.ipynb

```
group = {"Dana", "Omer", "Dana", "Lior"}
print(group)
print(len(group))
```

A WhatsApp group

Add Dana twice?
She is still in the group once.



A set: nobody is in twice

L05_02_sets.ipynb

```
group = {"Dana", "Omer", "Dana", "Lior"}
print(group)
print(len(group))
```

OUTPUT

```
{'Dana', 'Omer', 'Lior'}
3
```

A WhatsApp group

Add Dana twice? She is still in the group once.



A set: nobody is in twice

L05_02_sets.ipynb

```
group = {"Dana", "Omer", "Dana", "Lior"}
print(group)
print(len(group))
```

OUTPUT

```
{'Dana', 'Omer', 'Lior'}
3
```

A WhatsApp group

Add Dana twice? She is still in the group once.



- Curly brackets `{ }`. Duplicates disappear
- No order: your screen may show the names in another order

Drop the duplicates

```
signups = ["Dana", "Omer", "Dana", "Lior", "Dana", "Maya"]
students = set(signups)
print(len(signups))
print(len(students))
```

📋 Dana clicked “sign up” 3 times.
How many **real** students are there?

Drop the duplicates

```
signups = ["Dana", "Omer", "Dana", "Lior", "Dana", "Maya"]
students = set(signups)
print(len(signups))
print(len(students))
```

OUTPUT

```
6
4
```

📋 Dana clicked “sign up” 3 times.
How many **real** students are there?

Drop the duplicates

```
signups = ["Dana", "Omer", "Dana", "Lior", "Dana", "Maya"]
students = set(signups)
print(len(signups))
print(len(students))
```

OUTPUT

```
6
4
```

📋 Dana clicked “sign up” 3 times.
How many **real** students are there?

`set(...)` turns a list into a set: an instant duplicate remover.

add: put one more in

```
stickers = {"cat", "pizza"}
stickers.add("rocket")
stickers.add("cat")
print(stickers)
print(len(stickers))
```

add: put one more in

```
stickers = {"cat", "pizza"}
stickers.add("rocket")
stickers.add("cat")
print(stickers)
print(len(stickers))
```

OUTPUT

```
{'pizza', 'cat', 'rocket'}
3
```

add: put one more in

```
stickers = {"cat", "pizza"}
stickers.add("rocket")
stickers.add("cat")
print(stickers)
print(len(stickers))
```

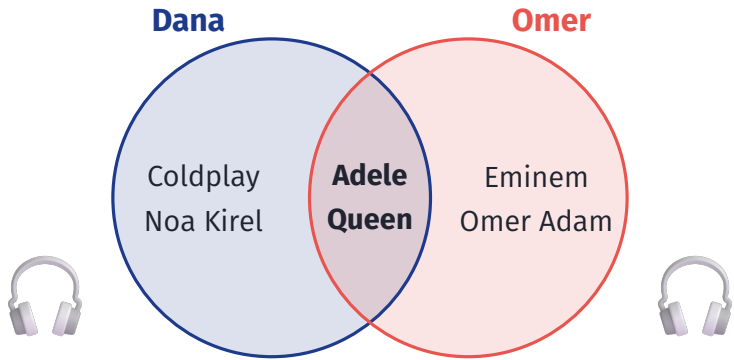
OUTPUT

```
{'pizza', 'cat', 'rocket'}
3
```

- Sets use `add`, lists use `append`
- Adding `"cat"` again does nothing. No error, no copy



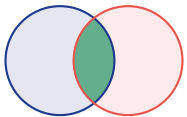
Two friends, two playlists



Which artists do they share? What goes on the party playlist?

&: what both like (intersection)

```
dana = {"Adele", "Coldplay", "Noa Kirel", "Queen"}
omer = {"Queen", "Eminem", "Adele", "Omer Adam"}
print(dana & omer)
```

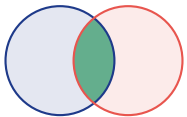


&: what both like (intersection)

```
dana = {"Adele", "Coldplay", "Noa Kirel", "Queen"}
omer = {"Queen", "Eminem", "Adele", "Omer Adam"}
print(dana & omer)
```

OUTPUT

```
{'Queen', 'Adele'}
```



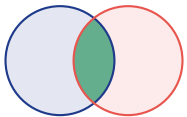
&: what both like (intersection)

```
dana = {"Adele", "Coldplay", "Noa Kirel", "Queen"}
omer = {"Queen", "Eminem", "Adele", "Omer Adam"}
print(dana & omer)
```

OUTPUT

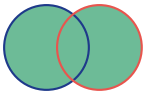
```
{'Queen', 'Adele'}
```

The middle of the two circles.
Intersection = “in both”.



|: everything (union)

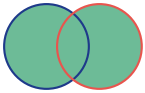
```
party = dana | omer
print(party)
print(len(party))
```



| is the “pipe” key, usually above Enter. Same dana and omer as before.

|: everything (union)

```
party = dana | omer
print(party)
print(len(party))
```



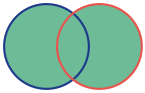
OUTPUT

```
{'Queen', 'Adele', 'Coldplay', 'Eminem', 'Noa Kirel', 'Omer Adam'}
6
```

| is the “pipe” key, usually above Enter. Same dana and omer as before.

|: everything (union)

```
party = dana | omer
print(party)
print(len(party))
```



Union = “in at least one”. The party playlist, with Adele only once.

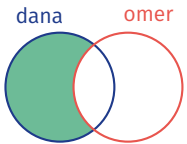
OUTPUT

```
{'Queen', 'Adele', 'Coldplay', 'Eminem', 'Noa Kirel', 'Omer Adam'}
6
```

| is the “pipe” key, usually above Enter. Same dana and omer as before.

–: only mine (difference)

```
print(dana - omer)
print(omer - dana)
```



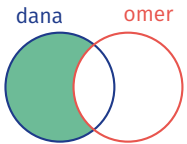
dana - omer

–: only mine (difference)

```
print(dana - omer)
print(omer - dana)
```

OUTPUT

```
{'Noa Kirel', 'Coldplay'}
{'Omer Adam', 'Eminem'}
```



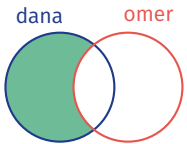
dana - omer

–: only mine (difference)

```
print(dana - omer)
print(omer - dana)
```

OUTPUT

```
{'Noa Kirel', 'Coldplay'}
{'Omer Adam', 'Eminem'}
```



dana - omer

Difference = “in mine, not in yours”.

Great for: songs to recommend to your friend.

List or set?

	List `[ ]`	Set `{ }`
Keeps order	✅	❌
Allows duplicates	✅	❌
Grab item by position	✅	❌
Compare two groups (&, |, −)	❌	✅

- Playlist → list
- Who signed up → set



Try it!

Try it in Colab notebooks/L05/L05_02_sets.ipynb

1. Make sets of 4 artists for you and a friend
2. Print what you share, and the party playlist
3. Turn `[3, 1, 3, 3, 2]` into a set. How many are left?



Predict the output

Predict the output!

```
a = {1, 2, 3, 3, 3}
b = {3, 4}
print(len(a))
print(a | b)
print(a & b)
```

Predict the output

Predict the output!

```
a = {1, 2, 3, 3, 3}
b = {3, 4}
print(len(a))
print(a | b)
print(a & b)
```

Answer

```
3
{1, 2, 3, 4}
{3}
```

The extra 3s vanish. Only 3 is in both.

Topic 3

Positions: indexing and slicing

Lecture 5 • Lists & Sets



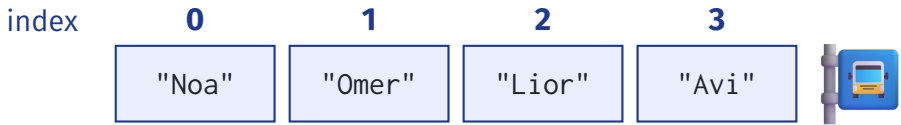
TODAY

1 Lists

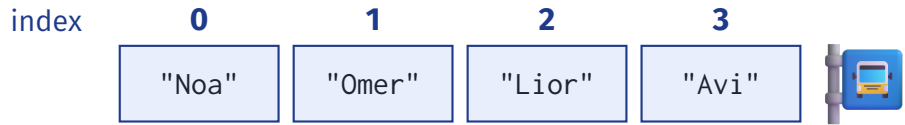
2 Sets

3 Positions: indexing and slicing

Every item has a position number

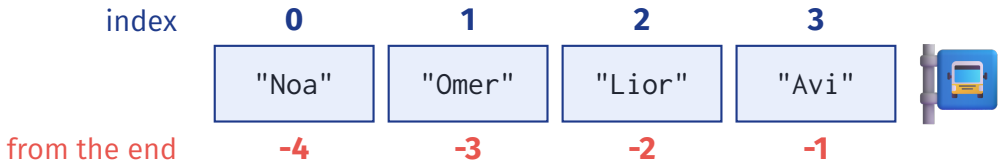


Every item has a position number



- Counting starts at **0**, not 1 (computers are weird)
- So 4 people sit in positions 0, 1, 2, 3

Every item has a position number



- Counting starts at **0**, not 1 (computers are weird)
- So 4 people sit in positions 0, 1, 2, 3
- Negative numbers count from the back: **-1** is the last

Joke break: I have three cats

```
cats = ["Mitzi", "Shuki", "Luna"]
print(cats[0], cats[2])
print(cats[3])
```

Joke break: I have three cats

```
cats = ["Mitzi", "Shuki", "Luna"]
print(cats[0], cats[2])
print(cats[3])
```

OUTPUT

```
Mitzi Luna
IndexError: list index out of range
```

Three cats: seats `0`, `1`, `2`.
There is no cat number `3`.



Why do programmers count from 0?

Because the first cat was already sitting there.

Grab one item: [ ]

L05_03_indexing_slicing.ipynb

```
queue = ["Noa", "Omer", "Lior", "Avi"]
print(queue[0])
print(queue[1])
print(queue[3])
```

Strings work the same way!

```
word = "PYTHON"
print(word[0])
print(word[3])
```

Grab one item: [ ]

L05_03_indexing_slicing.ipynb

```
queue = ["Noa", "Omer", "Lior", "Avi"]
print(queue[0])
print(queue[1])
print(queue[3])
```

OUTPUT

```
Noa
Omer
Avi
```

Strings work the same way!

```
word = "PYTHON"
print(word[0])
print(word[3])
```

Grab one item: [ ]

L05_03_indexing_slicing.ipynb

```
queue = ["Noa", "Omer", "Lior", "Avi"]
print(queue[0])
print(queue[1])
print(queue[3])
```

OUTPUT

```
Noa
Omer
Avi
```

Strings work the same way!

```
word = "PYTHON"
print(word[0])
print(word[3])
```

OUTPUT

```
P
H
```

Negative indexes: from the back

```
queue = ["Noa", "Omer", "Lior", "Avi"]
print(queue[-1])
print(queue[-2])
word = "PYTHON"
print(word[-1])
```

Negative indexes: from the back

```
queue = ["Noa", "Omer", "Lior", "Avi"]
print(queue[-1])
print(queue[-2])
word = "PYTHON"
print(word[-1])
```

OUTPUT

```
Avi
Lior
N
```

Negative indexes: from the back

```
queue = ["Noa", "Omer", "Lior", "Avi"]
print(queue[-1])
print(queue[-2])
word = "PYTHON"
print(word[-1])
```

OUTPUT

```
Avi
Lior
N
```

- `[-1]`: the last one, always
- No need to know the length



Slicing: a piece of the list

```
week = ["Sun", "Mon", "Tue", "Wed", "Thu", "Fri", "Sat"]
print(week[1:4])
print(week[0:5])
```

`[start:stop]`

From `start`, up to **but not including** `stop`.

Slicing: a piece of the list

```
week = ["Sun", "Mon", "Tue", "Wed", "Thu", "Fri", "Sat"]
print(week[1:4])
print(week[0:5])
```

OUTPUT

```
['Mon', 'Tue', 'Wed']
['Sun', 'Mon', 'Tue', 'Wed', 'Thu']
```

`[start:stop]`

From `start`, up to **but not including** `stop`.

Slicing: a piece of the list

```
week = ["Sun", "Mon", "Tue", "Wed", "Thu", "Fri", "Sat"]
print(week[1:4])
print(week[0:5])
```

OUTPUT

```
['Mon', 'Tue', 'Wed']
['Sun', 'Mon', 'Tue', 'Wed', 'Thu']
```

`week[0:5]`: the Israeli work week, Sun to Thu.

`[start:stop]`

From `start`, up to **but not including** `stop`.

Leave out start or stop

```
week = ["Sun", "Mon", "Tue", "Wed", "Thu", "Fri", "Sat"]
print(week[:3])
print(week[5:])
print(week[-2:])
```

Leave out start or stop

```
week = ["Sun", "Mon", "Tue", "Wed", "Thu", "Fri", "Sat"]
print(week[:3])
print(week[5:])
print(week[-2:])
```

OUTPUT

```
['Sun', 'Mon', 'Tue']
['Fri', 'Sat']
['Fri', 'Sat']
```

Leave out start or stop

```
week = ["Sun", "Mon", "Tue", "Wed", "Thu", "Fri", "Sat"]
print(week[:3])
print(week[5:])
print(week[-2:])
```

OUTPUT

```
['Sun', 'Mon', 'Tue']
['Fri', 'Sat']
['Fri', 'Sat']
```

- `[:3]`: from the start
- `[5:]`: to the end
- `[-2:]`: the last two



Slicing text

```
word = "chocolate"
print(word[:5])
print(word[5:])
```

```
email = "dana.levi@mail.tau.ac.il"
print(email[:9])
print(email[-6:])
```





Slicing text

```
word = "chocolate"
print(word[:5])
print(word[5:])
```

```
email = "dana.levi@mail.tau.ac.il"
print(email[:9])
print(email[-6:])
```

OUTPUT

```
choco
late
```





Slicing text

```
word = "chocolate"
print(word[:5])
print(word[5:])
```

OUTPUT

```
choco
late
```



```
email = "dana.levi@mail.tau.ac.il"
print(email[:9])
print(email[-6:])
```

OUTPUT

```
dana.levi
.ac.il
```



Steps: [start:stop:step]

```
week = ["Sun", "Mon", "Tue", "Wed", "Thu", "Fri", "Sat"]
print(week[::2])
print("stressed"[::-1])
print("level"[::-1])
```

Steps: [start:stop:step]

```
week = ["Sun", "Mon", "Tue", "Wed", "Thu", "Fri", "Sat"]
print(week[::2])
print("stressed"[::-1])
print("level"[::-1])
```

OUTPUT

```
['Sun', 'Tue', 'Thu', 'Sat']
desserts
level
```

Steps: [start:stop:step]

```
week = ["Sun", "Mon", "Tue", "Wed", "Thu", "Fri", "Sat"]
print(week[::2])
print("stressed"[::-1])
print("level"[::-1])
```

OUTPUT

```
['Sun', 'Tue', 'Thu', 'Sat']
desserts
level
```

- Step 2: every second item
- Step -1: backwards!

🎂 Exam week: *stressed* backwards is *desserts*.

Lists can change. Strings can't.

```
songs = ["Toxic", "Halo"]
songs[1] = "Yellow"
print(songs)
```

```
word = "cat"
word[0] = "b"
```

Lists can change. Strings can't.

```
songs = ["Toxic", "Halo"]
songs[1] = "Yellow"
print(songs)
```

OUTPUT

```
['Toxic', 'Yellow']
```

```
word = "cat"
word[0] = "b"
```

Lists can change. Strings can't.

```
songs = ["Toxic", "Halo"]
songs[1] = "Yellow"
print(songs)
```

OUTPUT

```
['Toxic', 'Yellow']
```

```
word = "cat"
word[0] = "b"
```

ERROR

```
TypeError: 'str' object does not
support item assignment
```

Lists can change. Strings can't.

```
songs = ["Toxic", "Halo"]
songs[1] = "Yellow"
print(songs)
```

OUTPUT

```
['Toxic', 'Yellow']
```

```
word = "cat"
word[0] = "b"
```

ERROR

```
TypeError: 'str' object does not
support item assignment
```

💡 Build a new string instead: `"b" + word[1:]` gives `"bat"`

Try it!

Try it in Colab notebooks/L05/L05_03_indexing_slicing.ipynb

1. Print the first and the last letter of your name
2. From a list of 7 days, print only the weekend
3. Is your name a palindrome? Compare it with `name[::-1]`





Predict the output

Predict the output!

```
nums = [10, 20, 30, 40, 50]
print(nums[1:3])
print(nums[-1])
print(nums[::-1])
```

Predict the output

Predict the output!

```
nums = [10, 20, 30, 40, 50]
print(nums[1:3])
print(nums[-1])
print(nums[::-1])
```

Answer

```
[20, 30]
50
[50, 40, 30, 20, 10]
```

Stop 3 is not included, so `40` is left out.

Bug Zoo: there is no seat number 3

BUG ZOO

```
queue = ["Noa", "Omer", "Lior"]
print(queue[3])
```

```
IndexError: list index out of range
```
ERROR



Off by one

3 people sit in seats 0, 1, 2. There is no seat 3.

Fix: `queue[2]`, or `queue[-1]` for the last one

Bug Zoo: things that aren't there

BUG ZOO

```
shopping = ["milk", "eggs"]
shopping.remove("pizza")
```

ERROR

```
ValueError: list.remove(x): x
not in list
```

Fix: check first with `if "pizza" in shopping:`

```
group = {"Dana", "Omer"}
print(group[0])
```

ERROR

```
TypeError: 'set' object is not
subscriptable
```

Fix: sets have no positions. Need a first item? Use a list.

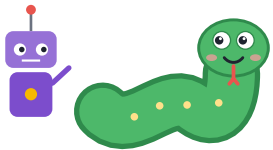
"Not subscriptable" = "you can't use `[ ]` on this". The intern is confused, not angry!

AI Corner: practice slicing

AI CORNER

A good prompt

"Explain Python slicing like `word[1:4]` to a beginner. Use 3 examples with the word `"falafel"`. Then quiz me with 2 questions."



Double-check

Run every example in Colab. AI often slips by one on the stop position!

Recap

✔ Today you learned

- A list `[ ]` keeps order: `append`, `insert`, `remove`, `pop`, `sort`
- A set `{ }` has no duplicates: `&` both, `|` all, `-` only mine
- Positions start at 0, `-1` is the last, `[start:stop]` slices



Next time: let the intern repeat itself

- Do something for **every** item in a list
- Keep asking until the password is right
- Sum, average and max, step by step





3D emoji: Microsoft Fluent Emoji (MIT License). Pip the snake: original drawing. Full list: images/CREDITS.md